

Trees

TRAVERSALS

Note : Visualize with 3 nodes

1. Inorder Iterative(Left-Root-Right)

- Initialize `current variable` with root, push left node until its null
- Pop last left node process it, then initialize current variable with right node

2. Preorder Iterative(Left-Root-Right)

- First add root to stack
- While stack is not empty pop from stack process the element, then push right node and then left node

3. Postorder Iterative(Left-Root-Right)

- Create two stack input and output
- Push root to a input stack, the pop from stack, then push the element to ouput stack
- Push left node to input stack and right node to input stack

BINARY SEARCH TREE (BST)

A Binary Search Tree is a node-based binary tree with a special ordering property that allows for fast lookups, insertions, and deletions.

Properties

- **BST Invariant:** For any given node `N` :
 - All values in its **left subtree** are **less than** `N` 's value.
 - All values in its **right subtree** are **greater than** `N` 's value.
 - Both its left and right subtrees must also be binary search trees.
- **No Duplicate Nodes:** A standard BST does not allow duplicate values.
- **In-order Traversal:** An in-order traversal of a BST yields its nodes' values in **sorted ascending order**.
- **Time Complexity:** For a balanced BST, operations like search, insertion, and deletion take **$O(\log n)$** time. In the worst case (a skewed or degenerate tree), they take **$O(n)$** time.

Operations

Insertion

To insert a value, you traverse the tree from the root. If the new value is less than the current node's value, you go left; otherwise, you go right. You continue until you reach a `null` spot, where you insert the new node.

```
TreeNode insert(TreeNode root, int key) {
    if (root == null) {
        return new TreeNode(key);
    }
    if (key < root.val) {
        // Recursively insert into the left subtree
        root.left = insert(root.left, key);
    } else if (key > root.val) {
        // Recursively insert into the right subtree
    }
}
```

```

        root.right = insert(root.right, key);
    }
    // Return the (possibly modified) root of the subtree
    return root;
}

```

Deletion

Deletion is more complex and handles three cases for the node to be deleted:

1. **No children (leaf node):** Simply remove the node.
2. **One child:** Replace the node with its child.
3. **Two children:** Find the node's **in-order successor** (the smallest value in its right subtree), replace the node's value with the successor's value, and then recursively delete the successor node.

```

TreeNode delete(TreeNode root, int key) {
    if (root == null) return null;

    if (key < root.val) {
        root.left = delete(root.left, key);
    } else if (key > root.val) {
        root.right = delete(root.right, key);
    } else { // Found the node to delete
        // Case 1 & 2: Node with one or no child
        if (root.left == null) return root.right;
        if (root.right == null) return root.left;

        // Case 3: Node with two children
        // Find the in-order successor (smallest value in the right subtree)
        TreeNode successor = findMin(root.right);
        root.val = successor.val; // Replace node's value with successor's
        root.right = delete(root.right, root.val); // Delete the successor
    }
    return root;
}

// Helper to find the minimum value node in a subtree
TreeNode findMin(TreeNode node) {
    while (node.left != null) {
        node = node.left;
    }
    return node;
}

```

AVL Tree

An **AVL Tree** (Adelson-Velsky & Landis) is a self-balancing BST where the **balance factor** of every node is in **{-1, 0, 1}**.

Balance Factor(node) = height(node.left) - height(node.right)

After every insertion or deletion, the tree checks balance factors bottom-up and applies **rotations** to restore balance. This guarantees **$O(\log n)$** for search, insert, and delete.

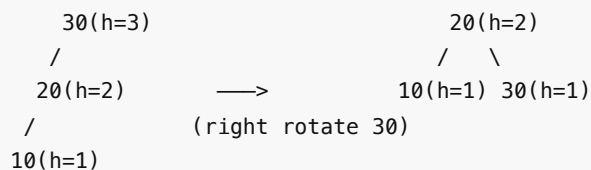
Four Rotation Cases:

Case	Trigger	Rotation
Left-Left (LL)	BF > 1 and inserted in left subtree of left child	Right Rotate
Right-Right (RR)	BF < -1 and inserted in right subtree of right child	Left Rotate
Left-Right (LR)	BF > 1 and inserted in right subtree of left child	Left Rotate on left child, then Right Rotate
Right-Left (RL)	BF < -1 and inserted in left subtree of right child	Right Rotate on right child, then Left Rotate

(h = height of node)

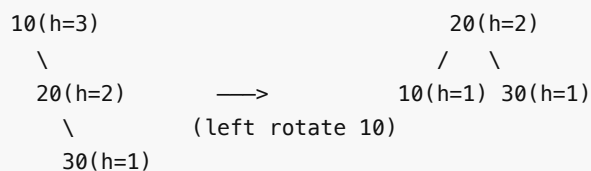
1. LL Case → Single Right Rotate

Insert 1, 2, 3 ... then insert triggers at 3



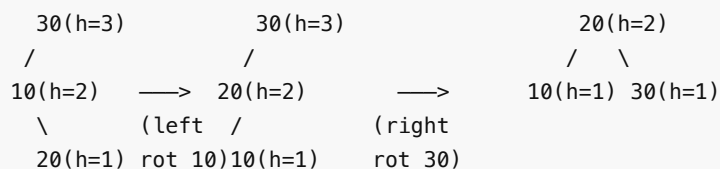
2. RR Case → Single Left Rotate

Insert 3, 2, 1 ... then insert triggers at 1



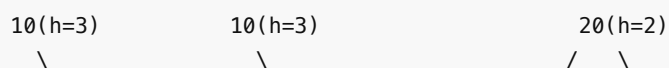
3. LR Case → Left Rotate on left child, then Right Rotate

Insert 30, 10, 20



4. RL Case → Right Rotate on right child, then Left Rotate

Insert 10, 30, 20



```

30(h=2)  —>  20(h=2)  —>  10(h=1) 30(h=1)
/      (right \      (left
20(h=1)  rot 30)  30(h=1)  rot 10)

```

```

public class AVLTree {

    private static class Node {
        int key, height;
        Node left, right;

        Node(int key) {
            this.key = key;
            this.height = 1; // new node is a leaf with height 1
        }
    }

    private Node root;

    // — Height & Balance Factor —————
    private int height(Node n) {
        return (n == null) ? 0 : n.height;
    }

    private int balanceFactor(Node n) {
        return (n == null) ? 0 : height(n.left) - height(n.right);
    }

    private void updateHeight(Node n) {
        n.height = 1 + Math.max(height(n.left), height(n.right));
    }

    // — Rotations —————
    //
    // Right Rotate (y):          Left Rotate (x):
    //      y              x              x              y
    //      / \      →      / \      / \      →      / \
    //      x  T3      T1  y      T1  y      x  T3
    //      / \      / \      / \      / \
    //      T1  T2      T2  T3      T2  T3      T1  T2
    //
    private Node rightRotate(Node y) {
        Node x = y.left;
        Node T2 = x.right;

        x.right = y;      // rotate
        y.left = T2;

        updateHeight(y);  // update y first (it's now lower)
        updateHeight(x);
        return x;         // new root of this subtree
    }
}

```

```

private Node leftRotate(Node x) {
    Node y = x.right;
    Node T2 = y.left;

    y.left = x;        // rotate
    x.right = T2;

    updateHeight(x);   // update x first (it's now lower)
    updateHeight(y);
    return y;          // new root of this subtree
}

// — Rebalance —————
// Called after insert/delete. Checks BF and applies the
// appropriate single or double rotation.
private Node rebalance(Node node) {
    updateHeight(node);
    int bf = balanceFactor(node);

    // Left-heavy
    if (bf > 1) {
        if (balanceFactor(node.left) < 0) {
            node.left = leftRotate(node.left); // LR case
        }
        return rightRotate(node);              // LL case
    }

    // Right-heavy
    if (bf < -1) {
        if (balanceFactor(node.right) > 0) {
            node.right = rightRotate(node.right); // RL case
        }
        return leftRotate(node);                // RR case
    }

    return node; // balanced
}

// — Insert —————
public void insert(int key) {
    root = insert(root, key);
}

private Node insert(Node node, int key) {
    if (node == null) return new Node(key);

    if (key < node.key)    node.left = insert(node.left, key);
    else if (key > node.key) node.right = insert(node.right, key);
    else                  return node; // duplicate keys not allowed

    return rebalance(node);
}

```

```

}

// — Delete —————
public void delete(int key) {
    root = delete(root, key);
}

private Node delete(Node node, int key) {
    if (node == null) return null;

    if (key < node.key)    node.left = delete(node.left, key);
    else if (key > node.key) node.right = delete(node.right, key);
    else {
        // Found node to delete
        if (node.left == null) return node.right;
        if (node.right == null) return node.left;

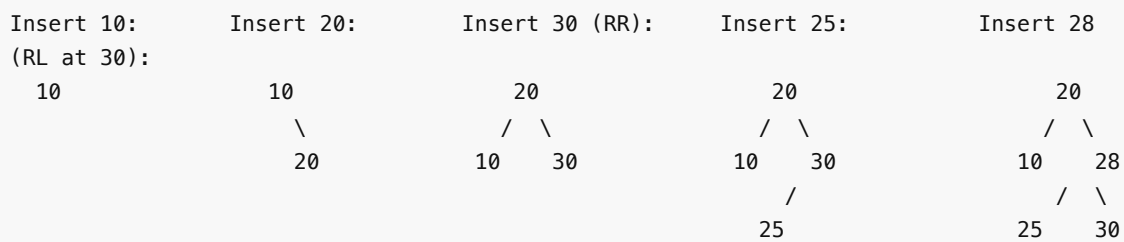
        // Two children: replace with in-order successor (smallest in right
subtree)
        Node successor = node.right;
        while (successor.left != null) successor = successor.left;
        node.key = successor.key;
        node.right = delete(node.right, successor.key);
    }

    return rebalance(node);
}

// — Search —————
public boolean search(int key) {
    Node curr = root;
    while (curr != null) {
        if (key == curr.key)    return true;
        else if (key < curr.key) curr = curr.left;
        else                   curr = curr.right;
    }
    return false;
}
}

```

Walkthrough — Insert 10, 20, 30, 25, 28:



Complexity:

Operation	Time	Space
Search	$O(\log n)$	$O(1)$ iterative
Insert	$O(\log n)$	$O(\log n)$ stack
Delete	$O(\log n)$	$O(\log n)$ stack

When to use: When you need guaranteed **$O(\log n)$** lookups and the dataset has frequent lookups relative to inserts/deletes. AVL trees are more strictly balanced than Red-Black trees, so lookups are slightly faster but insertions/deletions may be slightly slower due to more rotations.

RED-BLACK TREE

A **Red-Black Tree** is a self-balancing BST where each node stores an extra bit: its **color** (Red or Black). The coloring rules ensure the tree stays approximately balanced.

Properties (Invariants):

1. Every node is either **Red** or **Black**.
2. The **root** is always Black.
3. Every `null` leaf (NIL) is Black.
4. If a node is Red, **both its children must be Black** (no two consecutive reds).
5. Every path from a node to its descendant NIL leaves has the **same number of Black nodes** (black-height).

These rules guarantee: $h \leq 2 \cdot \log_2(n+1)$, so all operations are **$O(\log n)$** .

Fixing Violations After Insert (new node is always Red):

There are two types of fix-up operations:

Type 1: Recoloring (no structural change) When the uncle is Red, we just flip colors — no rotation needed. The grandparent becomes Red, so we move up and check again.

Type 2: Rotation + Recolor (structural change) When the uncle is Black, the tree is structurally unbalanced. We rotate to fix it.

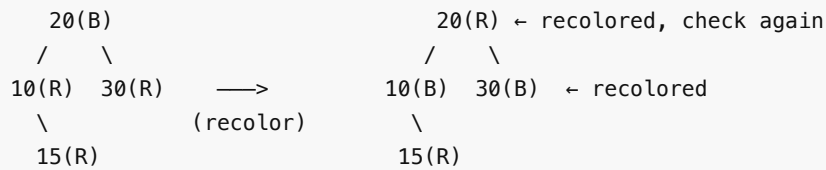
- If the new node is an **inner child** (zig-zag), we first rotate the parent to make it an outer child.
- Then we rotate the grandparent and recolor.

Case	Uncle Color	Fix Type	Action
1	Uncle is Red	Recoloring only	Recolor parent & uncle to Black, grandparent to Red. Move up.
2	Uncle is Black , node is inner child	Rotation	Rotate parent (transforms to Case 3)
3	Uncle is Black , node is outer child	Rotation + Recolor	Rotate grandparent + recolor parent/grandparent

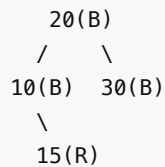
TYPE 1: RECOLORING (Uncle is Red)

No rotations. Just flip colors and propagate upward.

Example: Insert 15 into this tree



Then recolor root back to Black:

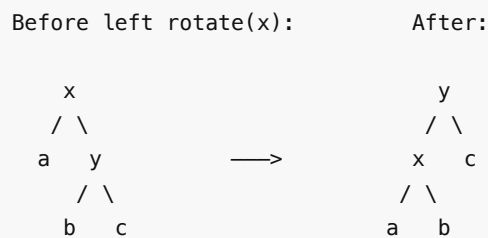


What happened:

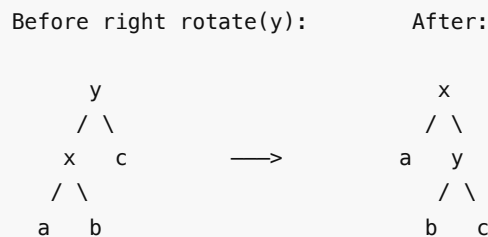
- Parent 10(R) and Uncle 30(R) → both flipped to Black
- Grandparent 20(B) → flipped to Red
- Root must be Black → recolor 20 back to Black
- No rotation needed!

TYPE 2: ROTATION (Uncle is Black)

Left Rotation: Pivots a node down-left, its right child takes its place.



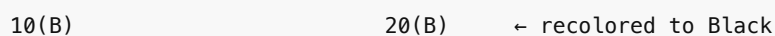
Right Rotation: Pivots a node down-right, its left child takes its place.

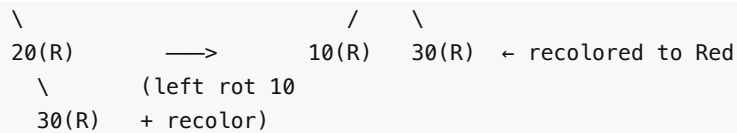


Case 3 — Outer child, Uncle Black → Single Rotation + Recolor

Example: Insert 10, 20, 30

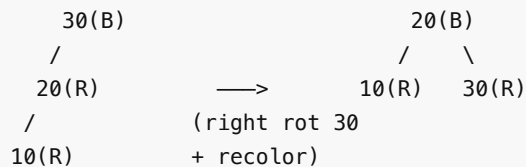
After inserting 30 → violation: 20(R)-30(R) consecutive reds, uncle is NIL(B)
30 is outer child (right-right) → left rotate grandparent 10 + recolor





Mirror example: Insert 30, 20, 10

10 is outer child (left-left) → right rotate grandparent 30 + recolor



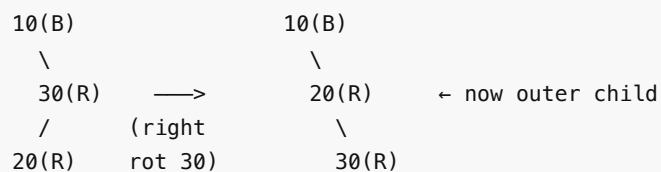
Case 2 → Case 3 — Inner child, Uncle Black → Double Rotation + Recolor

Example: Insert 10, 30, 20

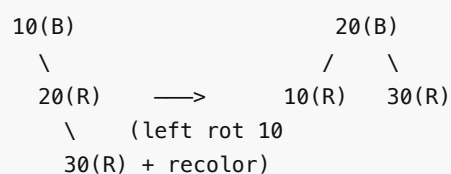
After inserting 20 → violation: 30(R)-20(R), uncle is NIL(B)

20 is inner child (right-left) → first right rotate parent 30, then left rotate grandparent 10

Step 1: Right rotate parent 30 (makes it outer child → Case 3)

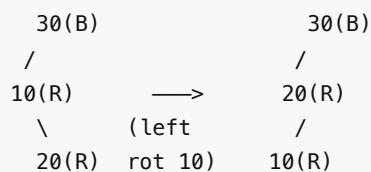


Step 2: Left rotate grandparent 10 + recolor (Case 3)

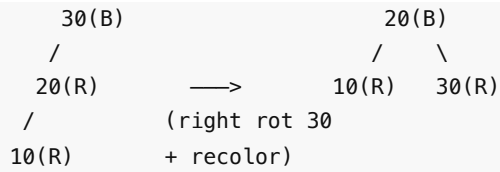


Mirror example: Insert 30, 10, 20

Step 1: Left rotate parent 10 (inner → outer)



Step 2: Right rotate grandparent 30 + recolor



```

public class RedBlackTree {

    private enum Color { RED, BLACK }

    private static class Node {
        int key;
        Color color;
        Node left, right, parent;

        Node(int key) {
            this.key = key;
            this.color = Color.RED; // new nodes are always Red
        }
    }

    private Node root;
    private final Node NIL; // sentinel for null leaves

    public RedBlackTree() {
        NIL = new Node(0);
        NIL.color = Color.BLACK;
        NIL.left = NIL.right = NIL.parent = NIL;
        root = NIL;
    }

    // — Rotations —————
    private void leftRotate(Node x) {
        Node y = x.right;
        x.right = y.left;
        if (y.left != NIL) y.left.parent = x;

        y.parent = x.parent;
        if (x.parent == NIL) root = y;
        else if (x == x.parent.left) x.parent.left = y;
        else x.parent.right = y;

        y.left = x;
        x.parent = y;
    }

    private void rightRotate(Node y) {
        Node x = y.left;
        y.left = x.right;
        if (x.right != NIL) x.right.parent = y;
    }
}
  
```

```

    x.parent = y.parent;
    if (y.parent == NIL)        root = x;
    else if (y == y.parent.left) y.parent.left = x;
    else                        y.parent.right = x;

    x.right = y;
    y.parent = x;
}

// — Insert —————
public void insert(int key) {
    Node z = new Node(key);
    z.left = z.right = z.parent = NIL;

    // Standard BST insert
    Node parent = NIL, curr = root;
    while (curr != NIL) {
        parent = curr;
        if (key < curr.key)    curr = curr.left;
        else if (key > curr.key) curr = curr.right;
        else                  return; // duplicate
    }
    z.parent = parent;
    if (parent == NIL)        root = z;
    else if (key < parent.key) parent.left = z;
    else                      parent.right = z;

    // Fix Red-Black violations
    insertFixup(z);
}

// — Insert Fixup —————
// We only violate Property 4 (red parent + red child).
// Walk up the tree, handling 3 symmetric cases per side.
private void insertFixup(Node z) {
    while (z.parent.color == Color.RED) {
        if (z.parent == z.parent.parent.left) {
            Node uncle = z.parent.parent.right;

            if (uncle.color == Color.RED) {
                // Case 1: Uncle is Red → recolor and move up
                z.parent.color = Color.BLACK;
                uncle.color = Color.BLACK;
                z.parent.parent.color = Color.RED;
                z = z.parent.parent;
            } else {
                if (z == z.parent.right) {
                    // Case 2: z is right child → left rotate to make it Case 3
                    z = z.parent;
                    leftRotate(z);
                }
                // Case 3: z is left child → right rotate grandparent + recolor
            }
        }
    }
}

```

```

        z.parent.color = Color.BLACK;
        z.parent.parent.color = Color.RED;
        rightRotate(z.parent.parent);
    }
} else {
    // Mirror: parent is right child of grandparent
    Node uncle = z.parent.parent.left;

    if (uncle.color == Color.RED) {
        z.parent.color = Color.BLACK;
        uncle.color = Color.BLACK;
        z.parent.parent.color = Color.RED;
        z = z.parent.parent;
    } else {
        if (z == z.parent.left) {
            z = z.parent;
            rightRotate(z);
        }
        z.parent.color = Color.BLACK;
        z.parent.parent.color = Color.RED;
        leftRotate(z.parent.parent);
    }
}
}
root.color = Color.BLACK; // ensure root is always Black
}

// — Search —————
public boolean search(int key) {
    Node curr = root;
    while (curr != NIL) {
        if (key == curr.key)    return true;
        else if (key < curr.key) curr = curr.left;
        else                  curr = curr.right;
    }
    return false;
}
}

```

Walkthrough — Insert 10, 20, 30, 15:

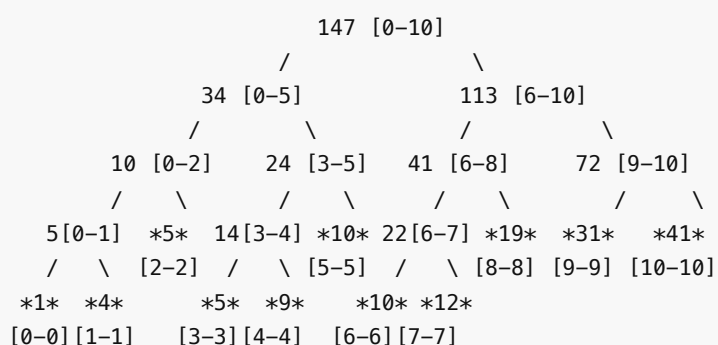
Insert 10 (root-Black): (Case 1):	Insert 20:	Insert 30 (Case 3):	Insert 15
10(B)	10(B)	20(B)	20(B)
	\	/ \	/ \
	20(R)	10(R) 30(R)	10(B)
30(B)			\
			15(R)

AVL vs Red-Black Tree:

Property	AVL Tree	Red-Black Tree
Balance	Strictly balanced ($BF \in \{-1,0,1\}$)	Approximately balanced ($h \leq 2 \log n$)
Rotations per insert	Up to $O(\log n)$	At most 2
Rotations per delete	Up to $O(\log n)$	At most 3
Lookup speed	Slightly faster	Slightly slower
Insert/Delete speed	Slightly slower	Slightly faster
Use case	Read-heavy workloads	Write-heavy workloads (Java TreeMap, Linux kernel)

SEGMENT TREE

Input Array: [1, 4, 5, 5, 9, 10, 10, 12, 19, 31, 41]



* = leaf node

How it works:

- Each node stores an aggregate (here: sum) of a contiguous subarray.
- The root covers the entire array $[0, n-1]$.
- Its left child covers $[0, \text{mid}]$, right child covers $[\text{mid}+1, n-1]$, and so on recursively until each leaf covers a single element.

Tree layout (stored in a flat array, 0-indexed):

- Node i 's left child = $2*i + 1$
- Node i 's right child = $2*i + 2$
- We allocate $4*n$ space to safely hold all nodes.

Time: Build $O(n)$, Query $O(\log n)$, Update $O(\log n)$

```
public class SegmentTree {
```

```

private final int[] tree; // internal array storing node values
private final int n;      // size of the original array

public SegmentTree(int[] arr) {
    n = arr.length;
    tree = new int[4 * n]; // 4*n guarantees enough space for any n
    buildTree(arr, 0, 0, n - 1);
}

// — BUILD —————
// Recursively construct the tree bottom-up.
//   node = index in tree[] for the current segment
//   start = left boundary of the segment this node covers
//   end   = right boundary of the segment this node covers
//
// Base case: leaf (start == end) → store the array element.
// Recursive: build left & right children, then merge (sum).
// —————
public void buildTree(int[] arr, int node, int start, int end) {
    if (start == end) {
        // Leaf: covers exactly one element arr[start]
        tree[node] = arr[start];
    } else {
        int mid = (start + end) / 2;
        buildTree(arr, 2 * node + 1, start, mid);    // left child
        buildTree(arr, 2 * node + 2, mid + 1, end);  // right child
        tree[node] = tree[2 * node + 1] + tree[2 * node + 2]; // merge
    }
}

// — QUERY —————
// Find the sum of elements in range [l, r].
//
// Three cases at each node covering [start, end]:
//   1. NO OVERLAP: [start, end] completely outside [l, r] → return 0
//   2. TOTAL OVERLAP: [start, end] completely inside [l, r] → return
tree[node]
//   3. PARTIAL OVERLAP: split into children and combine
// —————
public int query(int l, int r) {
    return query(0, 0, n - 1, l, r);
}

private int query(int node, int start, int end, int l, int r) {
    if (start > r || end < l) return 0; // no overlap
    if (start >= l && end <= r) return tree[node]; // total overlap

    int mid = (start + end) / 2;
    int leftSum = query(2 * node + 1, start, mid, l, r);
    int rightSum = query(2 * node + 2, mid + 1, end, l, r);
    return leftSum + rightSum;
}

```

```

// — POINT UPDATE —————
// Set arr[index] = value, then propagate changes up the tree.
//
// Walk from root toward the leaf that holds arr[index].
// At each level, go left or right depending on where index falls.
// Once at the leaf, set its value. On the way back up,
// recalculate each ancestor as the sum of its two children.
// —————
public void update(int index, int value) {
    update(0, 0, n - 1, index, value);
}

private void update(int node, int start, int end, int index, int value) {
    if (start == end) {
        tree[node] = value; // leaf node – update
        return;
    }

    int mid = (start + end) / 2;
    if (index <= mid) {
        update(2 * node + 1, start, mid, index, value); // go left
    } else {
        update(2 * node + 2, mid + 1, end, index, value); // go right
    }
    tree[node] = tree[2 * node + 1] + tree[2 * node + 2]; // recalculate
}
}

```

B-TREE

A **B-Tree** of order **m** is a self-balancing multi-way search tree optimized for systems that read/write large blocks of data (databases, file systems). Unlike binary trees, each node can hold **multiple keys** and have **multiple children**, minimizing disk I/O by keeping the tree height very small.

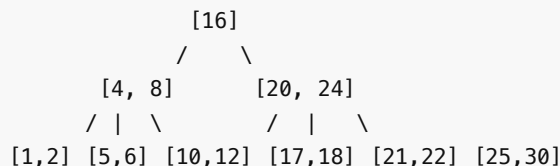
Properties of a B-Tree of order m:

Property	Rule
Max keys per node	$m - 1$
Max children per node	m
Min keys (non-root internal)	$\lceil m/2 \rceil - 1$
Min children (non-root internal)	$\lceil m/2 \rceil$
Root	At least 1 key (if non-empty)
Leaves	All at the same depth

Key ordering	Within a node, keys are sorted. Child i contains keys between key $i-1$ and key i .
--------------	---

Height: $h \leq \log(\lceil m/2 \rceil, (n+1)/2) \rightarrow$ very flat even for millions of entries.

B-Tree of order 3 (2-3 Tree)



Operations Overview:

- **Search:** Like BST search but at each node, scan through multiple keys to decide which child to follow. $O(\log n)$.
- **Insert:** Find the correct leaf. If the leaf is full ($m-1$ keys), **split** it: move the median key up to the parent. Splits may cascade up to the root, which is the only way the tree grows taller.
- **Delete:** Find and remove the key. If a node underflows (fewer than $\lceil m/2 \rceil - 1$ keys), fix by **borrowing** from a sibling or **merging** with a sibling.

```

public class BTree {

    private static final int ORDER = 3; // 2-3 tree (min degree t = 2)
    private static final int MAX_KEYS = ORDER - 1;
    private static final int MIN_KEYS = (ORDER + 1) / 2 - 1; // ceil(m/2) - 1

    private static class Node {
        int numKeys;
        int[] keys = new int[MAX_KEYS];
        Node[] children = new Node[ORDER];
        boolean isLeaf;

        Node(boolean isLeaf) {
            this.isLeaf = isLeaf;
        }
    }

    private Node root;

    public BTree() {
        root = new Node(true);
    }

    // — Search —————
    // At each node, find the first key ≥ target.
    // If found, return true. Otherwise, recurse into the child.
    public boolean search(int key) {
        return search(root, key);
    }
}

```

```

private boolean search(Node node, int key) {
    int i = 0;
    while (i < node.numKeys && key > node.keys[i]) i++;

    if (i < node.numKeys && key == node.keys[i]) return true;
    if (node.isLeaf) return false;
    return search(node.children[i], key);
}

// — Insert —————
// If root is full, split it first (tree grows one level).
// Then insert into the non-full tree.
public void insert(int key) {
    Node r = root;
    if (r.numKeys == MAX_KEYS) {
        Node newRoot = new Node(false);
        newRoot.children[0] = r;
        splitChild(newRoot, 0, r);
        root = newRoot;
        insertNonFull(newRoot, key);
    } else {
        insertNonFull(r, key);
    }
}

// Insert key into a node that is guaranteed not full.
// If leaf → shift keys right and insert.
// If internal → find correct child; split it if full, then recurse.
private void insertNonFull(Node node, int key) {
    int i = node.numKeys - 1;

    if (node.isLeaf) {
        while (i >= 0 && key < node.keys[i]) {
            node.keys[i + 1] = node.keys[i]; // shift right
            i--;
        }
        node.keys[i + 1] = key;
        node.numKeys++;
    } else {
        while (i >= 0 && key < node.keys[i]) i--;
        i++;
        if (node.children[i].numKeys == MAX_KEYS) {
            splitChild(node, i, node.children[i]);
            if (key > node.keys[i]) i++;
        }
        insertNonFull(node.children[i], key);
    }
}

// — Split Child —————
// Node y = parent.children[index] is full.

```

```

// Create a new node z, move the upper half of y's keys to z,
// promote the median key to parent.
private void splitChild(Node parent, int index, Node y) {
    Node z = new Node(y.isLeaf);
    int mid = MAX_KEYS / 2;

    // Move upper keys from y to z
    z.numKeys = MAX_KEYS - mid - 1;
    for (int j = 0; j < z.numKeys; j++) {
        z.keys[j] = y.keys[mid + 1 + j];
    }

    // Move upper children if not leaf
    if (!y.isLeaf) {
        for (int j = 0; j <= z.numKeys; j++) {
            z.children[j] = y.children[mid + 1 + j];
        }
    }

    y.numKeys = mid;

    // Shift parent's children/keys right to make room
    for (int j = parent.numKeys; j > index; j--) {
        parent.children[j + 1] = parent.children[j];
    }
    parent.children[index + 1] = z;

    for (int j = parent.numKeys - 1; j >= index; j--) {
        parent.keys[j + 1] = parent.keys[j];
    }
    parent.keys[index] = y.keys[mid]; // promote median
    parent.numKeys++;
}
}

```

Walkthrough — Insert 10, 20, 5, 30, 15 into B-Tree of order 3 (max 2 keys per node):

Step 1: Insert 10

[10]

Step 2: Insert 20 (room in node)

[10, 20]

Step 3: Insert 5 → node is full [5, 10, 20] → SPLIT

Median 10 promoted to new root, left=[5], right=[20]

```

    [10]
   /  \
  [5]  [20]

```

Step 4: Insert 30 → goes to right leaf [20] → room

```

      [10]
     /  \
    [5]  [20, 30]

```

Step 5: Insert 15 → goes to right leaf [15, 20, 30] → SPLIT
 Median 20 promoted to root, left=[15], right=[30]

```

      [10, 20]
     /  |  \
    [5] [15] [30]

```

Delete from B-Tree of order 3:

Starting tree:

```

      [10, 20]
     /  |  \
    [5] [15] [30]

```

Delete 15 → leaf [15] becomes empty → UNDERFLOW (min 1 key needed)
 Try borrowing from sibling [30] via parent key 20:
 Move parent key 20 down to empty node, move 30 up to parent.

```

      [10, 30]
     /  |  \
    [5] [20] [] ← wait, [30] is now empty!

```

Actually: borrow from right sibling through parent:
 Pull 20 down from parent, push 30 up to parent:

```

      [10, 30]
     /  |  \
    [5] [20] [] ← No! Merge instead.

```

MERGE: Merge [15]'s position with sibling [30] + parent key 20:
 Combine into [20, 30], remove key 20 from parent.

```

      [10]
     /  \
    [5]  [20, 30]

```

Delete 10 → 10 is in internal node → replace with in-order successor (20)
 Pull 20 from leaf, put 20 in root.

```

      [20]
     /  \
    [5]  [30]

```

Node Split Detail (order 3, max 2 keys):

Before split: node overflows with 3 keys

[A, B, C] (3 keys, max is 2 → must split)

After split: median B promoted to parent

```
Parent: [..., B, ...]
      /      \
    [A]      [C]
```

If parent also overflows → cascade split upward (tree grows taller)

Example cascade: Insert 25 into this tree

```
      [10, 20]                [10, 20, 30]                [20]
      / | \                  / | | \                  /  \
    [5] [15] [30]  →  [30] gets 25 → overflow  →      [10]   [30]
                                                    /  \   /  \
                                                    [5] [15] [25] [35]
```

split [25, 30] → promote 30
parent [10,20,30] → overflow!
split parent → promote 20

Complexity (B-Tree of order m, n keys):

Operation	Time
Search	$O(\log n)$
Insert	$O(\log n)$
Delete	$O(\log n)$
Height	$O(\log_m(n))$ — very flat

B+ TREE

A **B+ Tree** is a variation of a B-Tree with two key differences:

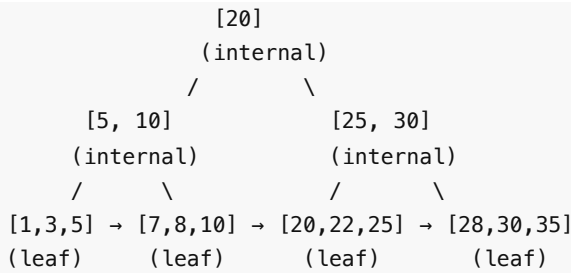
1. **All data lives in leaf nodes only.** Internal nodes store only keys as "road signs" for navigation.
2. **Leaf nodes are linked** in a doubly/singly linked list, enabling efficient range scans.

This is the data structure behind virtually all database indexes (MySQL InnoDB, PostgreSQL, SQLite).

Why B+ Tree over B-Tree for databases?

Feature	B-Tree	B+ Tree
Data location	Any node	Leaves only
Leaf linking	No	Yes (linked list)
Range queries	Must traverse tree	Sequential scan via leaf links
Internal node fan-out	Lower (keys + data)	Higher (keys only → more keys per node)
Cache/Disk efficiency	Good	Better (internal nodes fit more in memory)

B+ Tree



Leaves are linked: leaf1 → leaf2 → leaf3 → leaf4

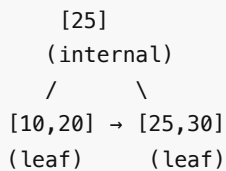
Key Operations:

- **Search:** Navigate internal nodes (like B-Tree) until you reach a leaf. All searches end at a leaf.
- **Range Query:** Find the starting leaf, then follow the linked list pointers — no need to re-traverse the tree.
- **Insert:** Insert into the correct leaf. If the leaf overflows, split it and **copy** the middle key up (not move, since data stays in leaves).
- **Delete:** Remove from the leaf. Handle underflow by borrowing or merging.

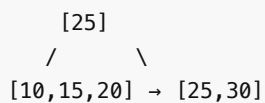
Insert into B+ Tree (order 4, max 3 keys per node):

Step 1: Insert 10, 20, 30 → all fit in one leaf (max 3)
 [10, 20, 30] (this is both root and leaf)

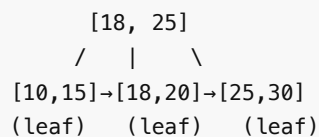
Step 2: Insert 25 → leaf overflows [10, 20, 25, 30] → SPLIT LEAF
 Split at middle: left=[10, 20], right=[25, 30]
 COPY middle key 25 up to new internal root (key stays in leaf!)



Step 3: Insert 15 → goes to left leaf [10, 15, 20] → fits



Step 4: Insert 18 → left leaf overflows [10, 15, 18, 20] → SPLIT LEAF
 Split: left=[10, 15], right=[18, 20]
 COPY 18 up to internal node.



B+ Tree vs B-Tree split difference:

B-Tree split (key MOVES up):

[10, 20, 30] → split

```
      [20]      ← 20 moved
     /   \
    [10]  [30]  ← 20 gone
```

B+ Tree split (key COPIES up):

[10, 20, 30] → split

```
      [20]      ← 20 copied
     /   \
    [10]  [20, 30] ← 20 stays in leaf!
```

Delete from B+ Tree (order 4, max 3 keys):

Starting tree:

```
      [18, 25]
     /  |  \
    [10,15]→[18,20]→[25,30]
```

Delete 20 → remove from leaf [18, 20] → becomes [18] → OK (min 1 key)

```
      [18, 25]
     /  |  \
    [10,15]→[18]→[25,30]
```

Delete 18 → remove from leaf [18] → becomes [] → UNDERFLOW!

Option 1: Borrow from left sibling [10, 15] → move 15 over.

Update parent separator from 18 to 15.

```
      [15, 25]
     /  |  \
    [10] → [15] → [25,30]
```

Delete 15 → leaf [15] becomes [] → UNDERFLOW!

Can't borrow (left sibling [10] has min keys).

MERGE: Combine [10] and [] with separator → [10]

Remove separator 15 from parent.

```
      [25]
     /   \
    [10] → [25,30]
```

Update in B+ Tree:

Update key 20 to 22:

1. Search for leaf containing 20
2. Replace 20 with 22 in the leaf
3. If 20 was a separator in an internal node, update it too

Before:

```
      [18, 25]
     /  |  \
    [10,15]→[18,20]→[25,30]
```

After:

```
      [18, 25]
     /  |  \
    [10,15]→[18,22]→[25,30]
```

^^ updated in leaf

(If 20 were a separator key in the parent, we'd update that too)

```

public class BPlusTree {

    private static final int ORDER = 4; // max children per internal node
    private static final int MAX_KEYS = ORDER - 1;

    // — Leaf Node —————
    // Stores actual key-value pairs. Linked to the next leaf.
    static class LeafNode {
        int numKeys;
        int[] keys = new int[MAX_KEYS];
        int[] values = new int[MAX_KEYS]; // data/record pointers
        LeafNode next; // pointer to next leaf for range scans
    }

    // — Internal Node —————
    // Stores only keys as separators (road signs).
    // children[i] covers keys < keys[i]; children[numKeys] covers keys ≥
    keys[numKeys-1].
    static class InternalNode {
        int numKeys;
        int[] keys = new int[MAX_KEYS];
        Object[] children = new Object[ORDER]; // InternalNode or LeafNode
    }

    private Object root; // can be InternalNode or LeafNode
    private LeafNode firstLeaf; // head of the leaf linked list

    public BPlusTree() {
        LeafNode leaf = new LeafNode();
        root = leaf;
        firstLeaf = leaf;
    }

    // — Search (exact key lookup) —————
    // Navigate internal nodes until we reach a leaf, then linear scan.
    public int search(int key) {
        LeafNode leaf = findLeaf(key);
        for (int i = 0; i < leaf.numKeys; i++) {
            if (leaf.keys[i] == key) return leaf.values[i];
        }
        return -1; // not found
    }

    // — Range Query —————
    // Find the first leaf containing startKey, then follow next pointers.
    public List<Integer> rangeQuery(int startKey, int endKey) {
        List<Integer> result = new ArrayList<>();
        LeafNode leaf = findLeaf(startKey);

        while (leaf != null) {
            for (int i = 0; i < leaf.numKeys; i++) {

```

```

        if (leaf.keys[i] >= startKey && leaf.keys[i] <= endKey) {
            result.add(leaf.values[i]);
        }
        if (leaf.keys[i] > endKey) return result;
    }
    leaf = leaf.next; // follow linked list
}
return result;
}

// Navigate from root to the leaf that would contain key
private LeafNode findLeaf(int key) {
    Object node = root;
    while (node instanceof InternalNode) {
        InternalNode internal = (InternalNode) node;
        int i = 0;
        while (i < internal.numKeys && key >= internal.keys[i]) i++;
        node = internal.children[i];
    }
    return (LeafNode) node;
}
}

```

Database Index Example:

SQL: SELECT * FROM users WHERE age BETWEEN 25 AND 35;

B+ Tree Index on 'age':

```

Internal:      [20 | 30 | 40]
              /   |   \
Leaves:  [15,18,20]→[22,25,28]→[30,32,35]→[38,40,45]
              ↑ start here           ↑ stop here

```

1. Navigate to leaf containing 25 → [22,25,28]
2. Scan forward via next pointers: 25, 28, 30, 32, 35 → done
3. No random I/O – all sequential reads!

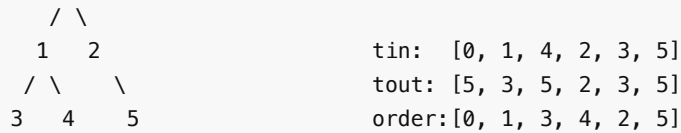
EULER TOUR TECHNIQUE

Flattens a tree into a linear array so that **subtree queries** become **range queries** on an array (solvable with Segment Tree / Fenwick Tree in $O(\log n)$).

Algorithm:

1. Run DFS from root. Record `tin[u]` (entry time) and `tout[u]` (exit time) for each node.
2. Build a flat array `order[]` where `order[i]` = node visited at time `i`.
3. Key insight: **subtree of node `u`** = contiguous range `[tin[u], tout[u]]` in the flat array.

Tree (rooted at 0):	DFS visit order:
	time: 0 1 2 3 4 5
0	node: 0→ 1→ 3→ 4→ 2→ 5



Subtree of 1 = order[tin[1]..tout[1]] = order[1..3] = {1, 3, 4} ✓
Subtree of 2 = order[tin[2]..tout[2]] = order[4..5] = {2, 5} ✓

Applications:

Problem	How Euler Tour Helps
Subtree sum/min/max	Range query on [tin[u], tout[u]] with Segment Tree
Ancestor check	u is ancestor of v iff tin[u] ≤ tin[v] && tout[u] ≥ tout[v]
LCA	RMQ on Euler tour between first occurrences of u and v
Path queries	Building block for Heavy-Light Decomposition

Java Implementation

```

public class EulerTour {

    private final List<List<Integer>> adj; // adjacency list of the tree
    private final int n;                  // number of nodes

    private int[] tin;    // tin[u] = entry time (when DFS first visits u)
    private int[] tout;   // tout[u] = exit time (when DFS finishes u's subtree)
    private int[] order;  // order[t] = which node has entry time t (the flat array)
    private int timer;    // global clock, incremented at each new visit

    public EulerTour(int n) {
        this.n = n;
        this.adj = new ArrayList<>();
        for (int i = 0; i < n; i++) adj.add(new ArrayList<>());
    }

    public void addEdge(int u, int v) {
        adj.get(u).add(v);
        adj.get(v).add(u);
    }

    // — Compute the tour —————
    // After this call:
    //   tin[u] = position in the flat array where node u appears
    //   tout[u] = last position belonging to u's subtree
    //   order[i] = the node at position i in the flat array
    //
    // Subtree of u → contiguous range [tin[u], tout[u]] in order[].
    // —————
    public void computeTour(int root) {

```

```

    tin = new int[n];
    tout = new int[n];
    order = new int[n];
    timer = 0;
    dfs(root, -1);
}

private void dfs(int u, int parent) {
    // Record entry time: u is the (timer)-th node we visit
    tin[u] = timer;
    order[timer] = u;
    timer++;

    // Visit all children (skip parent to avoid going back up)
    for (int v : adj.get(u)) {
        if (v != parent) {
            dfs(v, u);
        }
    }

    // Record exit time: all descendants of u have been visited
    tout[u] = timer - 1;
}

public int tin(int u) { return tin[u]; }
public int tout(int u) { return tout[u]; }
public int[] getOrder() { return order; }
}

```

Combining Euler Tour + Segment Tree for subtree queries:

```

// 1. Build the tree and compute Euler Tour
EulerTour et = new EulerTour(n);
// ... addEdge() calls ...
et.computeTour(root);

// 2. Build a flat array in Euler order: flat[i] = val[order[i]]
int[] flat = new int[n];
for (int i = 0; i < n; i++) {
    flat[i] = val[et.getOrder()[i]];
}

// 3. Build Segment Tree over the flat array
SegmentTree seg = new SegmentTree(flat);

// 4. Subtree sum of node u → range query [tin[u], tout[u]]
int subtreeSum = seg.query(et.tin(u), et.tout(u));

// 5. Update node u's value → point update at tin[u]
seg.update(et.tin(u), newValue);

```

```
// 6. Ancestor check: u is ancestor of v iff tin[u] <= tin[v] && tout[u] >= tout[v]
boolean isAncestor = (et.tin(u) <= et.tin(v) && et.tout(u) >= et.tout(v));
```

Walkthrough — Euler Tour + Segment Tree:

Node values: val = {1, 2, 3, 4, 5, 6} (same tree from above)
flat array (Euler order): [1, 2, 4, 5, 3, 6]

Subtree sum of node 1 → query(1, 3) → 2+4+5 = 11 ✓

Subtree sum of node 0 → query(0, 5) → 1+2+4+5+3+6 = 21 ✓

Is 0 ancestor of 4? → tin[0]=0 ≤ tin[4]=3 && tout[0]=5 ≥ tout[4]=3 → YES ✓